

LAPORAN PRAKTIKUM
STRUKTUR DATA
PEKAN KE 5

Disusun Oleh :

Muhammad Irfan 2511531008

Dosen Pengampu : Dr Wahyudi S.T M.T

Asisten Praktikum : Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2025

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum dari mata kuliah Struktur Data dengan judul Single Linked List

Laporan ini disusun sebagai bentuk hasil kegiatan praktikum Struktur Data yang di mana membahas tentang bahasa pemrograman java dan penggunaan Single linked list serta kegunaan masing-masing percabangan tersebut serta cara penerapannya.

Penulis menyadari bahwa laporan praktikum ini masih jauh dari sempurna baik dari segi penulisannya dan dari segi pembahasannya. Oleh karena itu saya ingin mengucapkan terima kasih kepada dosen pengampu, asisten praktikum yang telah membimbing serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, penulis ucapkan terima kasih pada dosen pengampu yang telah memberikan arahan dan semoga laporan ini dapat bermanfaat baik bagi penulis maupun pembaca dalam pemahaman tentang Single linked list.

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
1.Class NodeSSL_2511531008	2
Kode program	2
2. Class TambaSSL_2511531008.....	2
Kode program	2
Output	4
3.Class PencarianSSL_2511531008	5
Kode program	5
Ouput.....	6
4. HapusSSL_2511531008	6
kodeprogram	6
Output	9
BAB III PENUTUP	10
3.1 Kesimpulan	10
TUGAS PEKAN 4.....	11
1.Class Pasien_2511531008	11
2. Class RumahSakit_251153108	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan komponen yang paling penting untuk menyimpan nilai dan mengelola nilai secara baik dan efisien. Penyimpanan nilai dan data sangatlah penting dalam dunia pemrograman. Penyimpanan data array sangatlah terbatas. Ukurannya yang statis dan tidak bisa berkelanjutan membuat program menjadi lambat dan tidak efisien

Oleh karena itu, dibuatkan cara penyimpanan dan pengelolaan data dengan baik dengan benar. Salah satunya dengan single linked list. Single linked list merupakan struktur data yang menyimpan nilai dari node yang saling terhubung dalam satu arah. Setiap node menyimpan 1 nilai/data dan memiliki pointer ke node berikutnya. Hal ini penggunaan memori yang lebih fleksibel tanpa merombak seluruh struktur memori.

1.2 Tujuan Praktikum

Tujuan utama dari praktikum ini adalah memberikan pemahaman kepada mahasiswa tentang Konsep Single linked list serta implementasi di dalam pemrograman bahasa Java. Selain itu bagaimana pengaplikasian berbagai berbagai-macam problem dengan menggunakan Single linked list

1.3 Manfaat Praktikum

Manfaat utama dari praktikum ini adalah mahasiswa diharapkan dapat memahami konsep, operasi operasi dan mengimplementasikan dalam bahasa java. Selain itu, mahasiswa juga diharapkan untuk dapat mengolah dan meningkatkan analisis terhadap problem dengan menggunakan Single linked list.

BAB II

PEMBAHASAN

1. Class NodeSSL_2511531008

Kode program

```
1 package pekan5_2511531008;
2 import java.util.*;
3 public class NodeSSL_2511531008 {
4     //node bagian data
5     int data_1008;
6     //pointer ke node berikutnya
7     NodeSSL_2511531008 next_1008;
8     //konstruktor menginisialisasi node dengan data
9     public NodeSSL_2511531008 (int data_1008) {
10         this.data_1008 = data_1008;
11         this.next_1008 = next_1008;
12     }
13 }
```

Penjelasan singkat:

- Pembuatan kelas NodeSSL_2511531008 dengan atribut terdiri dari data, next dan attribute node
- Konstruktor node terdiri dari attribute data_1008 dan juga next_1008
- Next_1008 berfungsi sebagai pointer

2. Class TambaSSL_2511531008

Kode program

```
1 package pekan5_2511531008;
2 import java.util.*;
3 public class TambaSSL_2511531008 {
4     public static NodeSSL_2511531008 insertAtFront(NodeSSL_2511531008 head, int value) {
5         NodeSSL_2511531008 new_node_1008 = new NodeSSL_2511531008(value);
6         new_node_1008.next_1008 = head;
7         return new_node_1008;
8     }
9     // fungsi menambahkan node diakhir SLL
10    public static NodeSSL_2511531008 insertAtEnd_1008 (NodeSSL_2511531008 head, int value_1008) {
11        //buat sebuah node dengan sebuah nilai
12        NodeSSL_2511531008 newNode = new NodeSSL_2511531008(value_1008);
13        //jika list kosong maka node jadi head
14        if (head == null) {
15            return newNode;
16        }
17    }
```

```

16     }
17     // simpan head ke variabel sementara
18     NodeSLL_2511531008 last_1008 = head;
19     //telusuri ke node akhir
20     while(last_1008.next_1008 != null) {
21         last_1008 = last_1008.next_1008;
22     }
23     //ubah pointer
24     last_1008.next_1008 = newNode;
25     return head;
26 }
27 static NodeSLL_2511531008 GetNode(int data_1008) {
28     return new NodeSLL_2511531008(data_1008);
29 }
30 }

```

Penjelasan singkat :

- Membuat function insert Atfront berguna untuk menambahkan elemen baru sekaligus menjadi head baru
- Membuat objek node baru dengan nama newNode_1008. Kemudian mengarah pointer next dari node baru ke head yang lama
- Function insertAtEnd_1008 berguna untuk menambahkan elemen di posisi paling belakang
 - Jika list masih kosong head_1008 == null , maka node baru menjadi head
 - Jika tidak kosong, maka melakukan transversal pencarian hingga next bernilai null. Tidak ada node berikutnya.
 - Pointernya dipindahkna ke node yang baru

```

static NodeSLL_2511531008 insertPos_1008(NodeSLL_2511531008 headNode_1008,int position_1008, int value_1008) {
    NodeSLL_2511531008 head_1008 = headNode_1008;
    if(position_1008 < 1) System.out.print("invalid position");
    if(position_1008 == 1) {
        NodeSLL_2511531008 new_node = new NodeSLL_2511531008(value_1008);
        new_node.next_1008 = head_1008;
        return new_node;
    } else {
        while (position_1008-- != 0) {
            if (position_1008 == 1) {
                NodeSLL_2511531008 newNode = GetNode(value_1008);
                newNode.next_1008 = headNode_1008.next_1008;
                headNode_1008.next_1008 = newNode;
                break;
            }
        }
        headNode_1008 = headNode_1008.next_1008;
    }
    if(position_1008 != 1) System.out.print("posisi diluar jangkuan");
}
return head_1008;

```

```

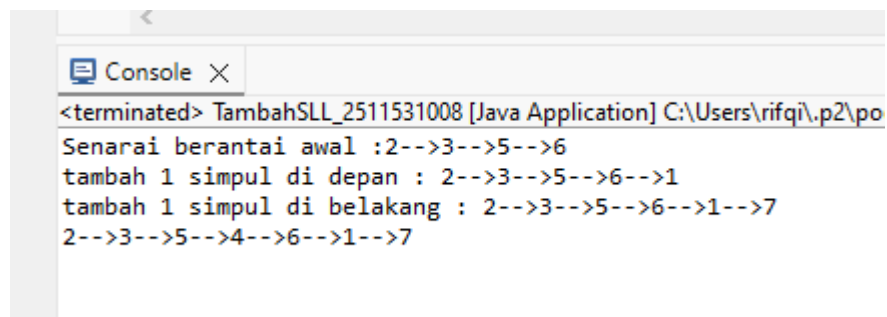
public static void printList(NodeSLL_2511531008 head_1008) {
    NodeSLL_2511531008 curr_1008 = head_1008;
    while (curr_1008.next_1008 != null) {
        System.out.print(curr_1008.data_1008 + "-->");
        curr_1008 = curr_1008.next_1008;
    }
    if(curr_1008.next_1008 == null ) {
        System.out.print(curr_1008.data_1008);
    }
    System.out.println();
}
}

```

Penjelasan singkat:

- Fungsi inserpos_1008 merupakan fungsi untuk menambahkan nilai pada posisi yang spesifik.
- Terdapat 3 paramter pada fungsi inserpos_1008
 - Jika position < 1, maka node tidak valid
 - Jika position == 1, maka node dibuat dan head diletakkan pada node yang lama
 - Jika selain itu, maka lakukan perulangan while mencari 1 posisi sebelum posisi target
 - Jika perulangan selesai tapi posisi tidak ditemukan (misal list hanya isi 3 tapi minta posisi 10), program akan mencetak posisi diluar jangkauan

Output



```

<terminated> TambahSLL_2511531008 [Java Application] C:\Users\rifqi\.p2\po
Senarai berantai awal :2-->3-->5-->6
tambah 1 simpul di depan : 2-->3-->5-->6-->1
tambah 1 simpul di belakang : 2-->3-->5-->6-->1-->7
2-->3-->5-->4-->6-->1-->7

```

3.Class PencarianSSL_2511531008

Kode program

```
1 package pekan5_2511531008;
2 import java.util.*;
3 public class PencarianSSL_2511531008 {
4     static boolean searchKey_1008(NodeSSL_2511531008 head_1008, int key_1008) {
5         NodeSSL_2511531008 curr_1008 = head_1008;
6         while(curr_1008 != null) {
7             if (curr_1008.data_1008 == key_1008) return true;
8             curr_1008 = curr_1008.next_1008;
9         }
10        return false;
11    }
12    public static void transversal(NodeSSL_2511531008 head_1008) {
13        //mulai dari head
14        NodeSSL_2511531008 curr_1008 = head_1008;
15        //telusuri sampai ponuiter null
16        while (curr_1008 != null) {
17            System.out.print(" " + curr_1008.data_1008);
18            curr_1008 = curr_1008.next_1008;
19            System.out.println();
20        }
21    }
22 }
```

Penjelasan singkat:

- Membuat function searchKey_1008. Fungsi ini mencari nilai/data yang diinginkan secara linear atau iteratif. Jika key_1008 == curr1008.data_1008, maka data ditemukan. Perulangan terus dilakukan selama curr_1008 tidak bernilai null
- Transversal adalah fungsi untuk mencetak semua elemen yang terdapat dalam single linked list

```
public static void main(String[] args) {
    NodeSSL_2511531008 head_1008 = new NodeSSL_2511531008(14);
    head_1008.next_1008 = new NodeSSL_2511531008(21);
    head_1008.next_1008.next_1008 = new NodeSSL_2511531008(30);
    head_1008.next_1008.next_1008.next_1008 = new NodeSSL_2511531008(10);
    System.out.print("Penelusuran SLL : ");
    transversal(head_1008);
    //data yang akan dicari
    int key_1008 = 30;
    System.out.print("cari data " + key_1008 + " = ");
    if(searchKey_1008(head_1008,key_1008)) {
        System.out.println("ketemu");
    }else {
        System.out.println("tidak ada");
    }
}
}
```

Penjelasan singkat:

- Fungsi utama ini mengisi nilai nilai pada single linked list , pencarian data dan menampilkan data
- Membuat objek single linked list dengan nama head_1008
- Kemudian mendefinisikan objek sekaligus mengisi nilai pada NodeSLL_2511531008
- Transversal(head_1008) adalah menampilkan semua elemen pada single linked list secara perbaris
- searchKey_1008(head_1008, key_1008). SearchKey_1008 mencari nilai key_1008 apakah nilai tersebut ada di dalam objek head_1008. Jika ada cetak ketemu, jika tidak print tidak ada

Ouput

```

-----
Penelusuran SLL : 14
21
30
10
cari data 30 = ketemu

```

4. HapusSSL_2511531008

kodeprogram

```

1 package pekan5_2511531008;
2 import java.util.*;
3 public class HapusSLL_2511531008 {
4     public static NodeSLL_2511531008 deleteHead_1008(NodeSLL_2511531008 head_1008) {
5         // jika SLL kosong
6         if (head_1008 == null) {
7             return null;
8         }
9         //pindahkan head ke node berikutnya
10        head_1008 = head_1008.next_1008;
11        return head_1008;
12    }
13    public static NodeSLL_2511531008 removeLatsNode_1008( NodeSLL_2511531008 head_1008) {
14        //jika list kosong , return null
15        if(head_1008 == null) {
16            return null;
17        }
18        //jika list satu node, hapus node dan return null
19        if(head_1008.next_1008 == null) {
20            return null;

```

```

24     NodeSLL_2511531008 secondlast_1008 = head_1008;
25     while(secondlast_1008.next_1008.next_1008 != null) {
26         secondlast_1008 = secondlast_1008.next_1008;
27     }
28     //hapus node terakhir
29     secondlast_1008.next_1008 = null;
30     return head_1008;
31 }

```

Penjelasan singkat:

- deleteHead_1008 merupakan fungsi untuk menghapus node pertama
 - jika head_1008 == null, maka return null
 - jika tidak maka pindahkan nilai head_1008.next_1008 ke head, kemudian di return nilai head_1008
- removelastNode_1008 merupakan fungsi untuk menghapus node terakhir pada single linked list
 - jika jika head_1008 == null, maka return null
 - jika list satu node, hapus node dan return null
 - jika berada 2 terakhir, kita harus berhenti di satu node sebelumnya. Dicek dengan menggunakan while secondlast.next.next.
 - setelah itu baru balikkan nilai head_1008

```

52 public static NodeSLL_2511531008 deleteNode_1008(NodeSLL_2511531008 head_1008, int position_1008) {
53     NodeSLL_2511531008 temp_1008 = head_1008;
54     NodeSLL_2511531008 prev_1008 = null;
55     //jika linked list null
56     if(temp_1008 == null) return head_1008;
57     //kasus 1 : head dihapus
58     if(position_1008 == 1) {
59         head_1008 = temp_1008.next_1008;
60         return head_1008;
61     }
62     //kasus 2 : menghapus node ditengah
63     //telusuri ke node yang dihapus
64     for(int i = 1; temp_1008 != null && i < position_1008 ; i++) {
65         prev_1008 = temp_1008;
66         temp_1008 = temp_1008.next_1008;
67     }
68     //jika ditemukan, hapus node
69     if(temp_1008 != null) {
70         prev_1008.next_1008 = temp_1008.next_1008;
71     }
72     }else {
73         System.out.println("Data tidak ada");
74     }
75     return head_1008;
76 }

```

Penjelasan singkat:

- Jika list null, maka mengembalikan nilai null, karena tidak mempunyai node satupun

- Jika head / node 1 hapus. Maka pindahkan head 1 ke head 2, dengan begitu node 1 keluar dari single linked list
- Jika menghapus node ditengah. Maka dilakukan pencarian dengan perulangan. Perulangan ini berjalan hingga variabel temp_1008 berada tepat pada node yang akan dihapus, sementara prev_1008 berada tepat di satu node sebelumnya
- Jika tidak ada node yang dihapuskan, maka

```

// ~~~~~
public static void printList(NodeSLL_2511531008 head_1008) {
    NodeSLL_2511531008 curr_1008 = head_1008;
    while(curr_1008.next_1008 != null) {
        System.out.print(curr_1008.data_1008 + "-->");
        curr_1008 = curr_1008.next_1008;
        if(curr_1008.next_1008 == null) {
            System.out.print(curr_1008.data_1008);
            System.out.println();
        }
    }
}

```

Penjelasan singkat:

- Fungsi ini mencetak semua nilai – nilai node secara berurutan dengan menggunakan pointer

```

72 public static void main(String[] args) {
73     // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6-> null
74     NodeSLL_2511531008 head_1008 = new NodeSLL_2511531008(1);
75     head_1008.next_1008 = new NodeSLL_2511531008(2);
76     head_1008.next_1008.next_1008 = new NodeSLL_2511531008(3);
77     head_1008.next_1008.next_1008.next_1008 = new NodeSLL_2511531008(4);
78     head_1008.next_1008.next_1008.next_1008.next_1008 = new NodeSLL_2511531008(5);
79     head_1008.next_1008.next_1008.next_1008.next_1008.next_1008 = new NodeSLL_2511531008(6);
80     //cetak list awal
81     System.out.println("list awal: ");
82     printList(head_1008);
83     //hapus head
84     head_1008 = deleteHead_1008(head_1008);
85     System.out.println("list setekah head dihapus: ");
86     printList(head_1008);
87     //hapus node terakhir
88     head_1008 = removeLatsNode_1008(head_1008);
89     System.out.println("list setelah simpul terakhir di hapus: ");
90     printList(head_1008);

91     //deleting node at position 2
92     int position_1008 = 2;
93     head_1008 = deleteNode_1008(head_1008,position_1008);
94     //print list after deletion
95     System.out.println("list setelah posisi 2 dihapus: ");
96     printList(head_1008);
97 }

```

Penjelasan singkat:

- Membuat objek single linked list dengan nama head_1008
- Kemudian objek head_1008 diisi dengan nilai – nilai seperti 1,2,3,4,5,6

- Baris 81 – 82 Cetak list node single linked list secara berantai
- Baris 84-86 menggunakan fungsi deleteHead_1008 yang berfungsi untuk menghapus node dari depan, kemudian cetak
- Baris 87-90 menggunakan fungsi removelastnode_1008 yang berfungsi untuk menghapus node dari belakang , kemudian cetak nilai setelah di hapus.
- Baris 91-96 menghapus nilai node dengan posisi secara spesifik., kemudian mencetak node SSL setelah di hapus

Output

```

Console x
<terminated> HapusSSL_2511531008 [Java Applicatio
list awal:
1-->2-->3-->4-->5-->6
list setekah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
list setelah posisi 2 dihapus:
2-->4-->5

```

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa single linked list merupakan struktur data linear yang menyimpan nilai dari node node yang saling terhubung dengan pointer next. Dengan begitu, single linked list lebih efisien daripada array karena lebih fleksibel, hemat memori tanpa merombak struktur penyimpanan memori. Selain itu, terdapat operasi – operasi pada single linked list seperti menyisipkan nilai dari depan, belakang, pencarian data serta menghapus data dari depan, belakang dan posisi spesifik. diharapkan mahasiswa dapat memahami konsep single linked list serta mengimplemtasikan kedalam kehidupan sehari hari.

TUGAS PEKAN 4

1.Class Pasien_2511531008

```
1 package pekan5_2511531008;
2
3 public class Pasien_2511531008 {
4     private String nama_1008;
5     private String penyakit_1008;
6     private int antrian_1008;
7     //pointer next
8     Pasien_2511531008 next_1008;
9     public Pasien_2511531008(String nama_1008, String penyakit_1008,int antrian_1008){
10         this.nama_1008 = nama_1008;
11         this.penakit_1008 = penyakit_1008;
12         this.antrian_1008 = antrian_1008;
13         this.next_1008 = next_1008;
14     }
15     public String getnama_1008(){
16         return nama_1008;
17     }
18     public String getpenyakit_1008(){
19         return penyakit_1008;
20     }
21
22     public String getnama_1008(){
23         return nama_1008;
24     }
25     public String getpenyakit_1008(){
26         return penyakit_1008;
27     }
28     public int getantrian_1008(){
29         return antrian_1008;
30     }
31
32     @Override
33     public String toString() {
34         return "No. Antrian: " + antrian_1008 +
35             " | Nama: " + nama_1008 +
36             " | Penyakit: " + penyakit_1008;
37     }
38 }
```

Penjelasan singkat:

- Membuat class dengan nama Pasien_2511531008
- Membuat konstuktur dengan atribute terdiri dari nama_1008,penyakit_1008 dan antrian_1008
- Terdapat getter dari masing masing attribue
- @override berfungsi untuk mencetak nilai attribute dari konstruktor Pasien_2511531008

2. Class RumahSakit_2511531008

```
3 public class RumahSakit_2511531008 {
4     static int counter = 0;
5     static Pasien_2511531008 head_1008 = null;
6     //1. tambah pasien didepan
7     public static Pasien_2511531008 InsertAtTail(String nama_1008,String penyakit_1008) {
8         counter++;
9         Pasien_2511531008 newnode_1008 = new Pasien_2511531008(nama_1008,penyakit_1008, counter);
10        // jika kosong
11        if(head_1008 == null) {
12            head_1008 = newnode_1008;
13            System.out.println("Data berhasil ditambahkan " + newnode_1008);
14            return newnode_1008;
15        }
16        Pasien_2511531008 last_1008 = head_1008;
17        while(last_1008.next_1008 != null) {
18            last_1008 = last_1008.next_1008;
19        }
20        //ubah pointer
21        last_1008.next_1008 = newnode_1008;
22        System.out.println("Data berhasil ditambahkan " + newnode_1008);
23        return newnode_1008;
24    }
25 }
```

Penjelasan :

- Deklarasi variabel counter dan objek pasien_2511531008 secara global scope.
- Fungsi insertAtTail berfungsi untuk menambahkan pasien secara FIFO
 - Baris 8 increment dari counter
 - Baris 9 membuat objek pasien_2511531008 dengan nama newnode_1008
 - Jika head_1008 kosong, maka jadikan newnode_1008 sebagai head. Kemudian cetak nilai newnode_1008 dan balikan nilai newnode_1008
 - Jika tidak maka lakukan perulangan while untuk menentukan pointer terakhir dari node. Kemudian baru diubah pointer lama jadi newnode_1008
 - Cetak nilai newnode_1008 dan balikkan nilai newnode_1008

```
--
26 //2. panggil pasien (delete)
27 public static Pasien_2511531008 panggilPasien_1008() {
28     if(head_1008 == null) return null;
29     else {
30         Pasien_2511531008 panggil_1008 = head_1008;
31         head_1008 = head_1008.next_1008;
32         System.out.println("Memanggil pasien: " + panggil_1008.getnama_1008());
33         return panggil_1008;
34     }
35 }
36 //3. tampilkan antrean
37 public static void listantrian_1008 () {
38     Pasien_2511531008 pasien_1008 = head_1008;
39     while(pasien_1008 != null) {
40         System.out.println(pasien_1008);
41         pasien_1008 = pasien_1008.next_1008;
42     }
43     System.out.println();
44 }
45 }
```

Penjelasan :

- Fungsi panggilPasien_1008 merupakan fungsi untuk menghapus node paling depan (FIFO)
 - Jika SSL tidak mempunyai head, maka return null
 - Jika tidak, maka nilai head_1008 disimpan dalam objek panggil_1008, kemudian di pointer head_1008 digeser ke head_1008 yang baru
 - Cetak nilai objek yang disimpan dalam panggil_1008

```

46 // 4. cari pasien
47 public static boolean pencarian_1008(String keyname_1008) {
48     Pasien_2511531008 pasien_1008 = head_1008;
49
50     while(pasien_1008 != null) {
51         if(keyname_1008.equalsIgnoreCase(pasien_1008.getnama_1008())) {
52             System.out.println("data ditemukan: " + pasien_1008.getnama_1008() );
53             return true;
54         }
55         pasien_1008 = pasien_1008.next_1008;
56     }
57     System.out.println("data tidak ditemukan");
58     return false;
59 }

60 // 5. cek total antrian
61 public static void cekStatusAntrian_1008() {
62     Pasien_2511531008 last_1008 = head_1008;
63     int total = 0;
64     if(head_1008 == null) {
65         System.out.println("tidak ditemukan");
66         return;
67     }
68     total++;
69     while(last_1008.next_1008 !=null) {
70         total++;
71         last_1008 = last_1008.next_1008;
72     }
73     System.out.print("jumlah total pasien = "+ total);
74     return;

```

Penjelasan :

- Fungsi pencarian_1008 mencari nilai keyname_1008 pada node pasien. Pencarian dilakukan dengan linear menggunakan while looping
 - Jika keyname ditemukan dalam node pasien tanpa memerhatikan kapital (ignore case) maka cetak ditemukan dan return true
 - Jika tidak , maka cetak data tidak ditemukan
- Fungsi cekstatusantrian_1008 menghitung jumlah antrean dengan perulangan while.
 - Jika head_108 == null atau tidak punya data, maka cetak tidak ditemukan
 - Jika mempunyai data maka hitung jumlah total data dengan perulangan


```

86 // display menu
87 public static void displaymenu_1008() {
88     System.out.println("\nAntrian Rumah Sakit NIM: 2511531008");
89     System.out.println("1. Daftarkan Pasien (Insert)");
90     System.out.println("2. Panggil Pasien (Delete Head)");
91     System.out.println("3. Tampilkan Antrian (Display)");
92     System.out.println("4. Cari Pasien (Search)");
93     System.out.println("5. Cek Status Antrian");
94     System.out.println("6. Keluar");
95     System.out.print("Pilihan: ");
96 }

```

Penjelasan:

- Fungsi ini menampilkan list menu pada output kode program

```

97 public static void main(String[] args) {
98     Scanner input = new Scanner(System.in);
99     int choice;
100     do {
101         displaymenu_1008();
102         choice = input.nextInt();
103         if(choice == 1) {
104             inputdata_1008(input);
105         }else if( choice == 2){
106             panggilPasien_1008();
107         }else if (choice == 3) {
108             listantrian_1008 ();
109         }else if (choice == 4) {
110             input.nextLine();
111             System.out.println("masukkan nama pasien yang diacri");
112             String cari = input.nextLine();
113             pencarian_1008(cari);
114         }else if (choice == 5) {
115             cekStatusAntrian_1008();
116         }else if(choice == 6) break;
117         else System.out.println("pilihan tidak valid");
118     }while(choice != 6);

```

Penjelasan:

- Input adalah objek dari scanner dan deklarasi int choice
- Terdapat perulangan do while untuk menampilkan displaymenu_1008() dan juga pilihan list dengan menginputkan nilai choice
- Jika nilai choice tidak sesuai dengan kriteria / selain 1-6, maka cetak pilihan tidak valid

Ouput

```

Antrian Rumah Sakit NIM: 2511531008
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: irfan
Masukkan penyakit: mag
Data berhasil ditambahkan No. Antrian: 1 | Nama: irfan | Penyakit: mag

Antrian Rumah Sakit NIM: 2511531008
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: rizki
Masukkan penyakit: diare
Data berhasil ditambahkan No. Antrian: 2 | Nama: rizki | Penyakit: diare

Antrian Rumah Sakit NIM: 2511531008
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
No. Antrian: 1 | Nama: irfan | Penyakit: mag
No. Antrian: 2 | Nama: rizki | Penyakit: diare

Antrian Rumah Sakit NIM: 2511531008
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil pasien: irfan

Antrian Rumah Sakit NIM: 2511531008
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
masukkan nama pasien yang diacri
irfan
data tidak ditemukan

Antrian Rumah Sakit NIM: 2511531008
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
jumlah total pasien = 1

```

Antrian Rumah Sakit NIM: 2511531008

1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 6

r